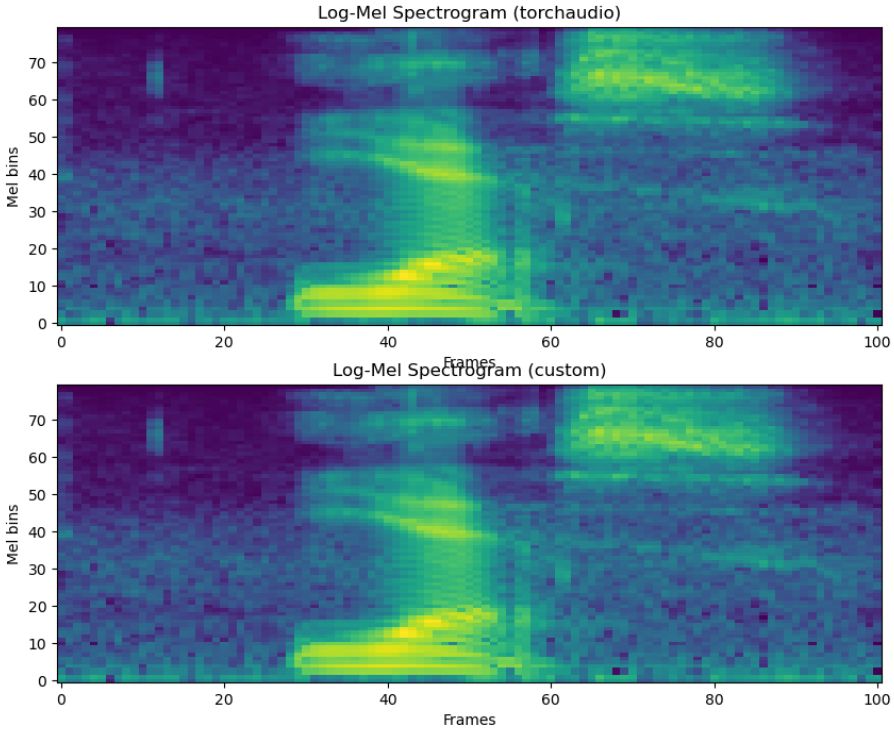
Домашняя работа №1.
2026_AITH: Speech recognition and generation

Попов Игорь Владленович

Task 1: Реализация LogMelFilterBanks

Код класса реализован в файле: **hw_1/melbanks.py**
Код тестирования класса находится в файле **tests/hw_1/test_logmel.py**

Сравнение визаулиаций спектрограмм полученных с помощью библиотеки torchaudio и кастовой реализации. Аудио файл: **data/1aeef15e_nohash_0.wav**



Task 2: Пайплайн обучения

Скрипт обучения - файл **hw_1/main.py**

Model:

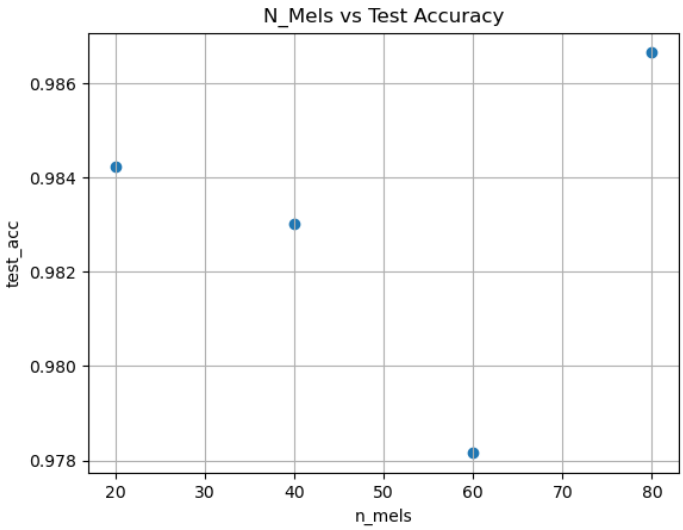
(conv1): Conv1d(n_mel, 64, kernel_size=(5,), stride=(1,), padding=(2,))
(conv2): Conv1d(64, 64, kernel_size=(5,), stride=(1,), padding=(2,))
(pool): AdaptiveAvgPool1d(output_size=1)
(classifier): Linear(in_features=64, out_features=1, bias=True)

Модель имеет примерно 46K параметров при n_mels=80

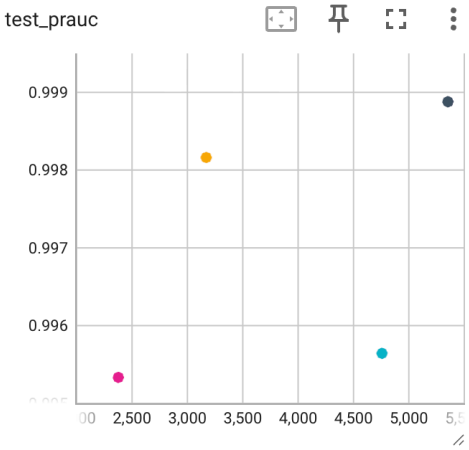
Реализация:
PyTorch Lightning, EarlyStopping, Adam Optimiser (lr=1e-3), No scheduler, BCEWithLogits loss, 50 эпох, batch size = 32, аугментации не использовались.

Железо:
Обучение проводилось на MacBook Pro M4 GPU MPS

Task 3: Эксперименты с числом мелбанков

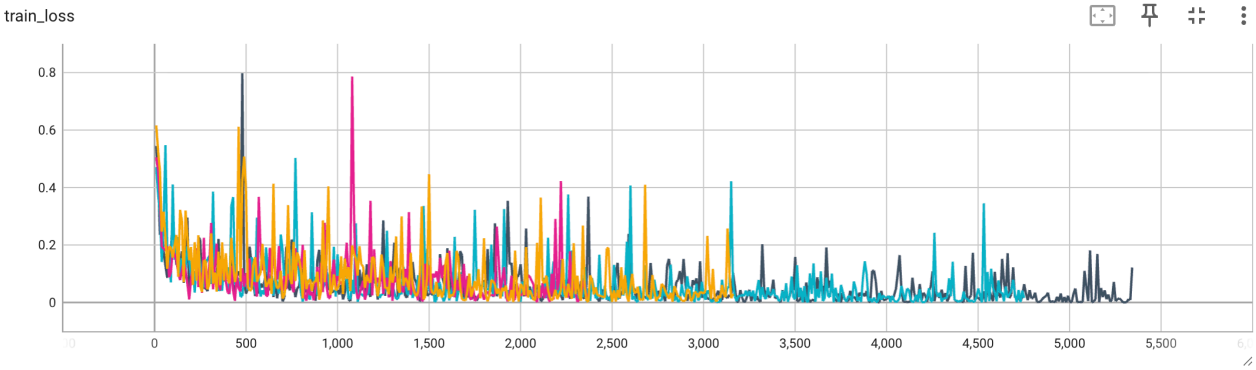


Accuracy на тесте

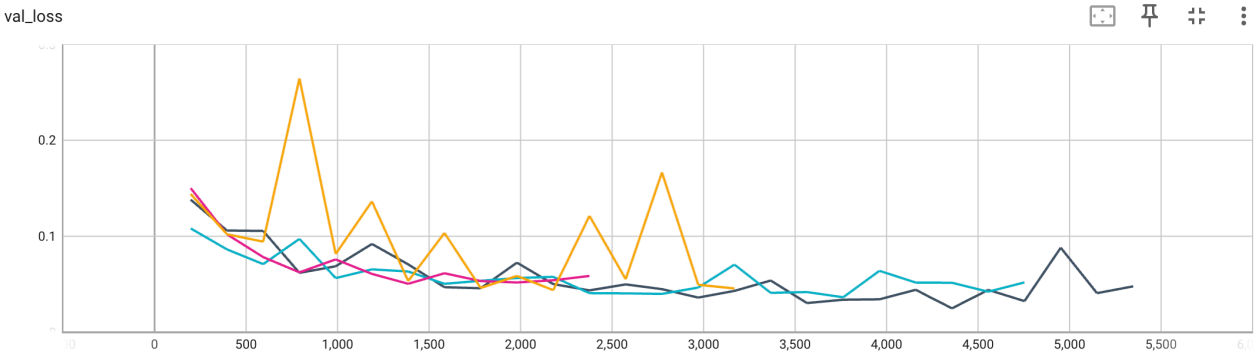


Run ↑	Value	Step	Relative
n_mels=20	0.9989	5,346	0
n_mels=40	0.9956	4,752	0
n_mels=60	0.9953	2,376	0
n_mels=80	0.9982	3,168	0

PR AUC на тесте



Run ↑	Value	Step	Relative
n_mels=20	0.1216	5,339	13.06 min
n_mels=40	0.0157	4,749	11.61 min
n_mels=60	0.0303	2,369	5.54 min
n_mels=80	0.0298	3,159	7.588 min



Run ↑	Value	Step	Relative
n_mels=20	0.0478	5,345	13 min
n_mels=40	0.0519	4,751	11.59 min
n_mels=60	0.0586	2,375	5.522 min
n_mels=80	0.0456	3,167	7.566 min

Метрика Accuracy зависит от выбранного параметра confidence threshold, корректнее подбирать этот порог на валидационной выборке, однако в проведенных экспериментах test accuracy вычислялась с фиксированным значением порога 0.5. Поэтому прямое сравнение моделей по этой метрике без настройки порога будет не совсем корректным. Поэтому для сравнения более информативной метрикой является PR AUC, которая не зависит от порога.

test_acc	test_prauc	flops(M)	params(K)	train_time	n_mels
0.9842233061790466	0.9988784790039062	2.694	27.01	0:00:29.926854	20
0.9830096960067749	0.9956435561180115	3.334	33.41	0:00:29.952842	40
0.9781553149223328	0.9953341484069824	3.974	39.81	0:00:29.584003	60
0.9866504669189453	0.9981616735458374	4.614	46.21	0:00:29.879969	80

При n_mels = 20, PR AUC - достигает максимального значения. Вероятно это связано с тем, что:

1. Задача yes/no простая, датасет небольшой. При небольшом количестве мелбанков больше частот объединяются в один бин. Таким образом мы **сглаживаем** потенциальные **шумы**. Это может работать как **регуляризация**, за счет уменьшения размерности признаков таким образом
2. Мы используем модель на основе Conv1d. Число мелбанков определяет размер входных каналов. Сеть простая, состоит из всего трех слоев. Вероятно, такая архитектура **эффективнее** работает с **меньшим размером входа**, а также **растет** ее **обобщающая способность**

Task 4. Эксперименты с числом groups в Conv1d слое

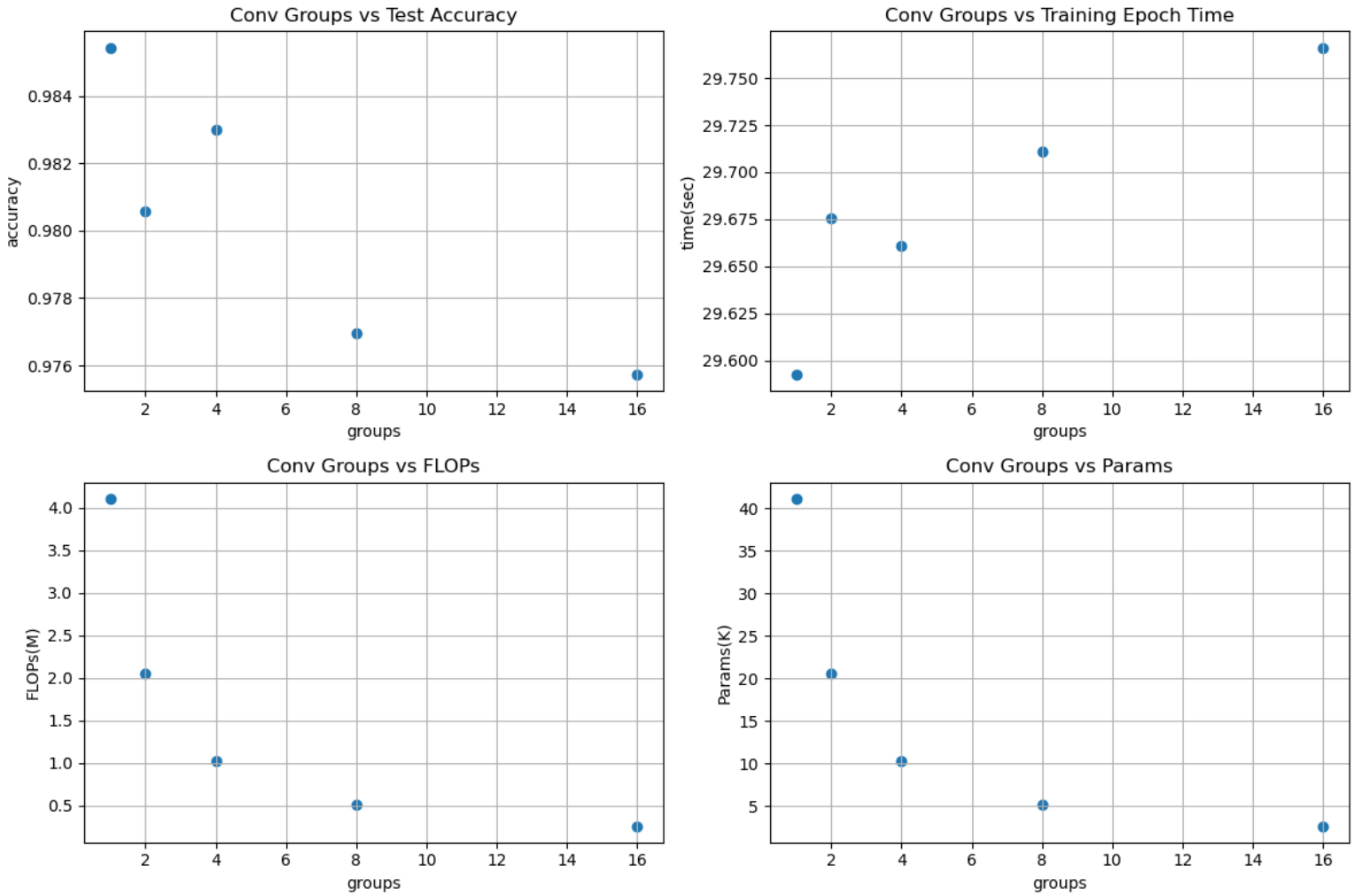
test_acc	test_prauc	flops(M)	params(K)	train_time	groups
0.9854369163513184	0.9979603886604309	4.102	41.09	0:00:29.592662	1
0.9805825352668762	0.9966850280761719	2.054	20.61	0:00:29.675874	2
0.9830096960067749	0.997607409954071	1.03	10.37	0:00:29.660821	4
0.9769417643547058	0.9948014616966248	0.518	5.25	0:00:29.711192	8
0.9757281541824341	0.9957648515701294	0.262	2.69	0:00:29.766133	16

При увеличении числа групп количество параметров и число FLOPs уменьшается
Вместо одной большой свертки с некоторым числом каналов считаем несколько сверток с меньшим числом каналов.
Число параметров обычной свертки: **Cout * Cin * K + Cout**
Число параметров grouped свертки: **(Cout * Cin * K) / g + Cout**

При увеличении числа групп точность падает
Как по Accuracy так и по PR AUC.

При увеличении числа среднее время эпохи обучения растет
Вероятно это связано с особенностями железа (GPU MPS, MacBook Pro M4) и размером сети, **накладные расходы** на вызов нескольких более мелких операций свертки вместо одной большой и **ухудшение** эффективности **векторизации нивелирует** преимущество в количестве операций FLOPs.

Таким образом **groups=1** показывает **максимальную точность**, но мы платим за это большим размером модели. **Компромиссом** по точность/вес является **groups=4**, при которой число параметров сокращается почти в x4 раза и незначительно проседают метрики. Более заметные закономерности стоит анализировать на более крупных моделях.


Conv Groups vs Test Accuracy
accuracy
0.984
0.982
0.980
0.978
0.976
2 4 6 8 10 12 14 16
groups
Conv Groups vs Training Epoch Time
time(sec)
29.750
29.725
29.700
29.675
29.650
29.625
29.600
2 4 6 8 10 12 14 16
groups
Conv Groups vs FLOPs
FLOPs(M)
4.0
3.5
3.0
2.5
2.0
1.5
1.0
0.5
2 4 6 8 10 12 14 16
groups
Conv Groups vs Params
Params(K)
40
35
30
25
20
15
10
5
2 4 6 8 10 12 14 16
groups